PLATFORM ENGINEERING · TECHNICAL ROADMAP

GardenWise / RootVio *Cloud Platform* Roadmap

A staged, 22-week plan to take the current GitHub Pages + hand-provisioned AWS estate to a fully codified platform: Terraform infrastructure-as-code, hardened GitHub Actions delivery, containerised workloads on Amazon EKS, and first-class observability with Prometheus and Grafana.

Terraform | GitHub Actions + OIDC | Amazon EKS | Prometheus & Grafana | SLO-driven ops

REPOSITORY
`github.com/imhero2k/GardenWise`

BASELINE COMMIT
`main @ dc4e305`

LIVE SURFACE
rootivio.app · ap-southeast-2

DOCUMENT
v1.0 · 10 September 2026

CONTENTS

What this document covers

How to read this roadmap

Each phase has a fixed structure: what it delivers, the week-by-week activity, the code or configuration that embodies it, and a definition of done expressed as things that can be demonstrated rather than claimed. Every phase ends at a numbered gate, and the next phase does not begin until that gate's exit criteria are met.

The current-state section is deliberately blunt about what is missing. Every absence recorded in section 02 was verified directly against the repository at commit `dc4e305`, because a roadmap built on an optimistic reading of the codebase is worse than no roadmap at all.

01 · EXECUTIVE SUMMARY

The application is healthy; the platform underneath it is undocumented

GardenWise (shipped as **RootVio**) is a React 19 + Vite 8 single-page app with an Express API that runs both locally and as an AWS Lambda, backed by PostgreSQL/PostGIS on RDS and a DynamoDB table for garden-planner layouts. The frontend has a genuinely good delivery story: every push to `main`, `iteration1` or `iteration2` builds and publishes to GitHub Pages behind the custom domain `rootivio.app`.

Everything below the frontend is the problem. There is **no infrastructure-as-code anywhere in the repository** — no Terraform, CloudFormation, SAM or CDK. API Gateway, both Lambda functions, the RDS instance and the DynamoDB table exist only as console-created resources and hard-coded URLs. The backend has no automated deployment at all: shipping an API change means hand-building a zip. There are zero automated tests, and observability consists of one `/api/health` endpoint plus `console.log`. A Docker and LocalStack/EKS packaging attempt was merged and then reverted in commits `6be8279` → `dc4e305`, so that work is recoverable but not active.

0	0	1 of 2	22
IAC FILES IN REPO	AUTOMATED TESTS	TIERS WITH CD	WEEKS TO TARGET

What the five phases deliver

PHASE	WEEKS	OUTCOME
0 · Baseline	1–2	The repository can be trusted: protected branches, rescued documentation, a complete configuration inventory, and a keyless path from CI into AWS
1 · Terraform	3–6	The production estate becomes reviewable and rebuildable, and a staging environment exists for the first time
2 · Actions	5–8	Tests actually gate merges, and the API deploys itself instead of being uploaded by hand
3 · EKS	9–14	Web and API run as scanned, signed containers on a cluster that Terraform can destroy and rebuild
4 · Prometheus	13–18	Failures are detected by alerts rather than by users, and incidents can be diagnosed after the fact
5 · Hardening	17–22	Backups are proven by restore, the auth migration finishes, and cost is governed rather than discovered

> **If only three things get done.** Terraform-import the existing estate, build the integration test suite with an automated API deploy, and ship `prom-client` with four alert rules. Those three — roughly nine weeks of work — remove every *critical* gap in section 03, and are worth more than the whole of Phases 3 and 5 combined.

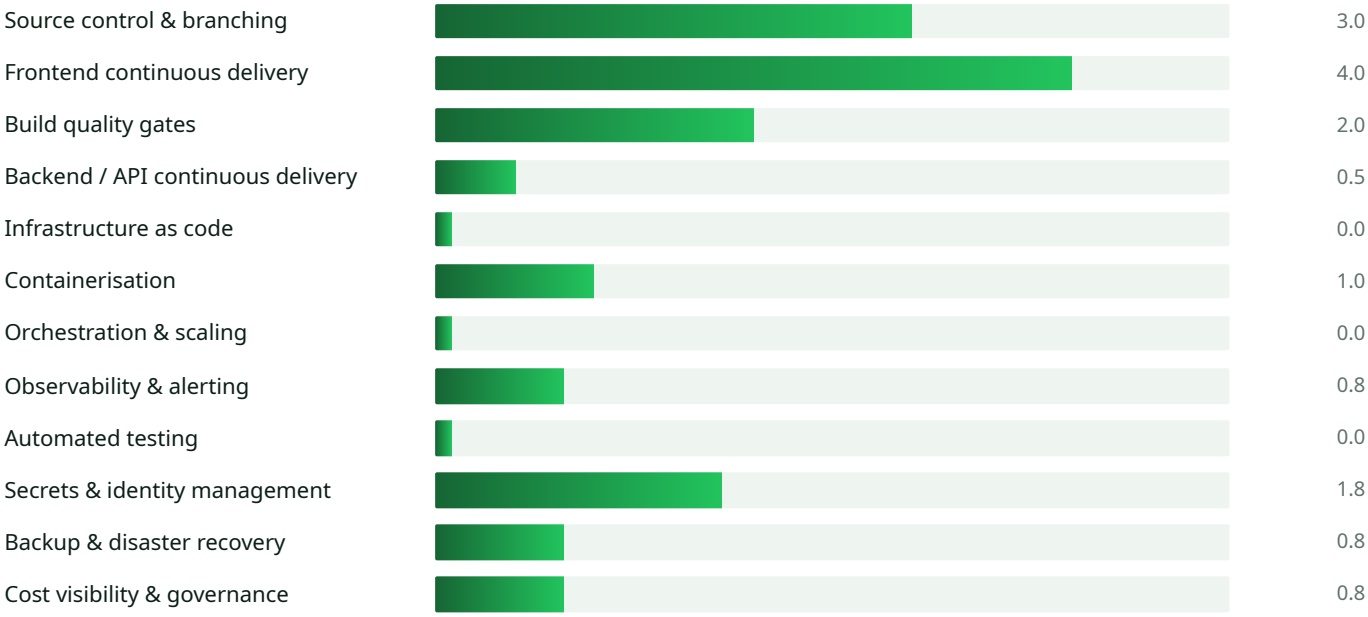
Verified inventory

Every row below was confirmed against `main @ dc4e305`. Absences are stated as explicitly as presences, because the absences are what the roadmap has to fund.

LAYER	WHAT IS ACTUALLY THERE	STATE
Frontend	React 19.2, Vite 8, React Router 7, TypeScript 5.9. SPA fallback via a `postbuild` step copying `index.html` → `404.html`. Leaflet map and a Three.js 3D planner. Source: `package.json`, `vite.config.ts`.	SOLID
API	A single Express 4 app exposing 10 routes, with dual entry points: `server/index.mjs` for local development and `server/lambda.mjs` through `@vendia/serverless-express`. Source: `server/app.mjs`.	WORKS
Relational data	PostgreSQL with PostGIS through a `pg` pool on `DATABASE_URL`. Tables `bioregion`, `plant`, `bioregion_plant`, `weed_info` and `plant_trait`, with ETL under `database/`.	WORKS
Key-value data	DynamoDB planner layouts, partition key `userId`, sort key `layoutId`. The table name comes from `DYNAMODB_PLANNER_LAYOUT_TABLE`, but the table itself was created by hand and is tracked nowhere.	UNTRACKED
Auth	Firebase Auth (Google, Apple, email) on the client, with Firebase Admin ID-token verification guarding the two planner-layout routes. Cognito work exists only on the `cognito-migration` branch.	IN FLUX
ML inference	A Python 3.11 ONNX weed classifier packaged as a container Lambda, with model weights tracked in `lambda_weedscan/`.	WORKS
CI / CD	`pages.yml` runs `npm ci`, `typecheck`, `lint` and `build` for three branches and publishes to Pages. `jekyll-gh-pages.yml` is a deliberate no-op. There is no backend deploy job.	PARTIAL
Infrastructure as code	None. No `.tf`, no CloudFormation, no SAM, no CDK, no `serverless.yml`. The API Gateway identifiers `tcal5txtn8` and `7muezhf0bl` appear as literals in source and CI.	ABSENT
Containers	Only `lambda_weedscan/Dockerfile`. A root `Dockerfile`, `Dockerfile.api`, `docker-compose.yml` and `nginx.conf` were added and then reverted in `dc4e305`.	REVERTED
Orchestration	No cluster and no manifests. `k8s/` and `scripts/deploy-localstack-eks.sh` were removed by the same revert.	ABSENT
Observability	`GET /api/health` reports database, planner-storage and Firebase Admin status. Otherwise `console.*`. No metrics endpoint, no Prometheus, no tracing, no error tracking.	MINIMAL
Testing	No test runner in `package.json`, no `*.test.*` files, no end-to-end suite, no coverage. The README's reference to `npm test` is stale.	ABSENT
Secrets	Seven `VITE_FIREBASE_*` values plus a PlantNet key held as GitHub secrets. Server secrets such as `DATABASE_URL` and the service-account JSON live in operator-managed Lambda environment variables.	AD HOC

Platform maturity, scored 0–5

Scores reflect what can be demonstrated today, not what is intended. The two dimensions sitting at zero are the ones that make every other number fragile.



Weighted mean: **1.3 / 5**. The target on completion of Phase 5 is **3.8 / 5** — deliberately not 5, since several dimensions such as multi-region disaster recovery and chaos engineering are out of scope for a project of this size and would cost more than they return.

Branch topology worth resolving

BRANCH	SIGNAL
`cognito-migration`	Real, active work: an auth abstraction, an Amplify Cognito provider, and `server/cognitoAuth.mjs` doing dual Cognito/Firebase verification. Not yet merged.
`migratingAuth`	Misleading name. The tip is an old frost-alert commit with no migration code in it. Rename or delete.
`origin/production`	Carries a `netlify.toml` that contradicts the GitHub Pages deploy on `main`.
`cloudflare/workers-autoconfig` ×3	Three near-duplicate branches proposing Cloudflare Workers hosting. Pick one or drop all three.
Dangling `107129f`	Holds the security plan and peer-coding COP. Reachable from no branch, so one `git gc` from permanent loss.

The ten gaps, ordered by what they block

Severity reflects blast radius today, not effort. The *Blocks* column is why the phase ordering in section 05 is the way it is.

#	GAP	CONSEQUENCE TODAY	SEVERITY	BLOCKS
1	**No infrastructure as code**	The production estate cannot be reviewed, diffed, rebuilt or reasoned about. A deleted API Gateway stage is an unrecoverable outage, and nobody can answer "what changed?"	CRITICAL	P1–P5
2	**No backend deployment pipeline**	API releases are manual zip uploads from a laptop. No provenance, no rollback, no repeatability, and the artefact depends on one machine's `node_modules`.	CRITICAL	P3
3	**No automated tests**	`typecheck` and `lint` cannot catch a broken SQL query, a bad DynamoDB key or a regressed auth guard. Every deploy is an untested deploy.	CRITICAL	P2, P3
4	**No metrics or alerting**	Failures are discovered by users. There is no latency, error-rate, saturation or pool-exhaustion signal, so no incident can be diagnosed after the fact.	CRITICAL	P4, P5
5	**Long-lived AWS credentials**	Deployment relies on static keys held outside the repository. There is no short-lived, auditable, per-workflow identity.	HIGH	P1, P2
6	**Endpoints hard-coded in source**	`tcal5txtn8` and `7muezhf0bl` are baked into `vite.config.ts`, `src/lib/predict.ts` and `pages.yml`, so a stack rebuild forces a source edit and there is no clean staging environment.	HIGH	P1, P2
7	**Unfinished auth migration**	Firebase on `main`, Cognito on a side branch, and a misleadingly named `migratingAuth` branch with no migration in it. Two identity providers to secure and no cut-over date.	HIGH	P1, P5
8	**Reverted container work**	Docker, compose and `k8s/` manifests were written and then discarded, so the effort has been spent but the benefit is not banked.	MEDIUM	P3
9	**Architecture docs unreachable**	The security plan and peer-coding COP live in dangling commit `107129f`, reachable from no branch and one `git gc` from permanent loss.	MEDIUM	P0
10	**Divergent hosting strategies**	GitHub Pages on `main`, Netlify on `production`, Cloudflare Workers on three `cloudflare/*` branches. Three answers to one question.	MEDIUM	P3

Two things to do before anything else

Sequencing rule. Do not create new AWS resources by hand during Phase 1. Every resource added outside Terraform after the import baseline widens the drift the project is trying to close, and unmanaged drift is the single most common reason infrastructure-as-code adoptions stall halfway and get abandoned.

Do this in week one. Rescue commit `107129f` before it is garbage-collected: `git branch rescue/docs 107129f`, then open a pull request. The security plan is the only written record of the intended AWS topology, and it is the reference input for the Terraform import work in Phase 1.

Guiding principles

Codify before you change

Import the estate exactly as it is and reach a clean `terraform plan` with no diff. Only then refactor. Mixing discovery with redesign is how a weekend disappears into a broken API Gateway.

One environment shape

Staging and production differ only by variable values, never by structure. If a change works in staging it works in production, which is the entire point of having a staging environment at all.

Instrument before you migrate

Metrics land in the Express app in Phase 4's first week, before any traffic moves to EKS, so the migration has a real before-and-after baseline rather than a hunch.

What each gap costs to close

GAP	PHASE	EFFORT	THE SINGLE HIGHEST-VALUE MOVE
1 · IaC	Phase 1	8 person-weeks	Declarative `import` blocks reviewed in a pull request, iterated to a zero-diff plan
2 · Backend CD	Phase 2	2 person-weeks	Build the artefact in CI and deploy to staging on every merge to `main`
3 · Tests	Phase 2	3 person-weeks	Supertest against a real PostGIS service container covers all 10 routes
4 · Metrics	Phase 4	1 person-week	`prom-client` histogram middleware plus a `/metrics` endpoint
5 · Credentials	Phase 0	0.5 person-weeks	GitHub OIDC provider with a narrowly scoped `sub` condition
6 · Hard-coded endpoints	Phase 0–1	0.5 person-weeks	Terraform outputs feeding build arguments, so identifiers stop living in source
7 · Auth migration	Phase 5	3 person-weeks	Dual-verify window gated on a per-provider failure metric
8 · Containers	Phase 3	2 person-weeks	Cherry-pick `6be8279` rather than starting from a blank page
9 · Lost docs	Phase 0	1 hour	One `git branch` command, this week
10 · Hosting	Phase 1–3	1 person-week	Commit to CloudFront and delete the alternatives

Reference topology on completion of Phase 4

Every box below is created and owned by Terraform. Nothing in this diagram is provisioned by hand, and nothing in it is addressed by a hard-coded identifier.

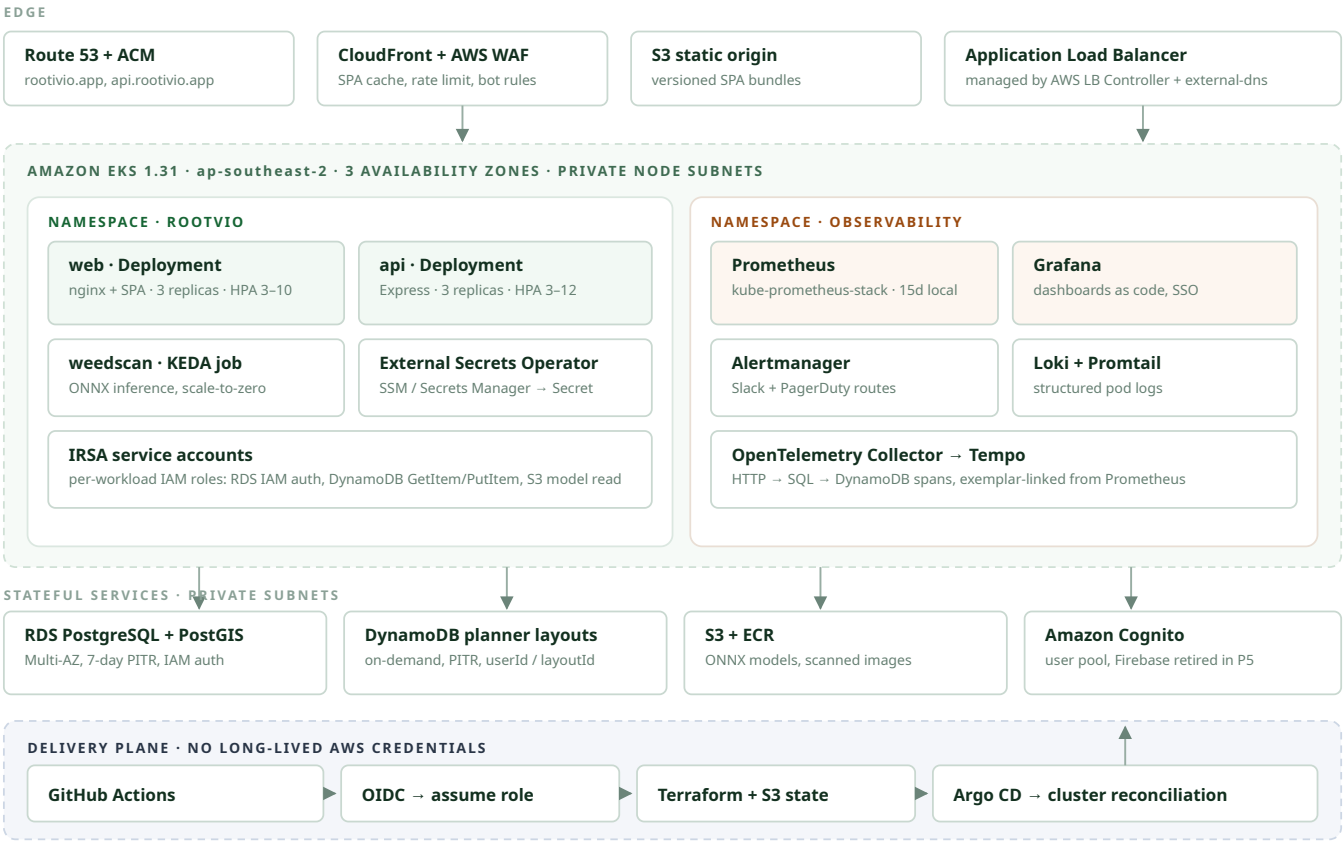


Figure 1 — Target topology. The delivery plane at the bottom holds no static AWS keys: GitHub Actions assumes a role through OIDC, Terraform owns AWS resources, and Argo CD reconciles cluster state from the repository.

What changes for a developer

TASK	TODAY	AFTER PHASE 4
Ship an API change	Build a zip locally, upload through the console	Merge a pull request; the image builds, Argo CD rolls it out, metrics confirm
Add an AWS resource	Click through the console, tell nobody	A Terraform module change with a reviewed `plan` on the pull request
Diagnose a slow request	Read `console.log` in CloudWatch	Grafana latency panel, exemplar, Tempo trace, the slow SQL statement
Roll back	Find the previous zip, hope it still works	`kubectl rollout undo`, or revert the pull request
Stand up an environment	Not feasible	`terraform apply -var-file=staging.tfvars`

05 · ROADMAP TIMELINE

Six workstreams, 22 weeks, five milestone gates

Phases overlap deliberately. Terraform work continues while pipelines are being built, and observability instrumentation starts inside the EKS phase so the cut-over is measured rather than hoped for.

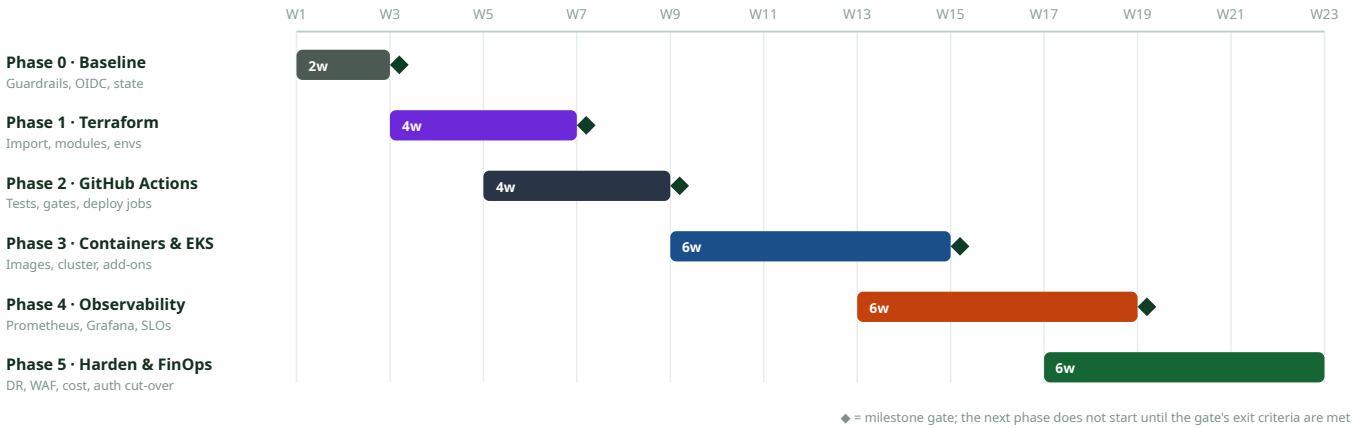


Figure 2 — Phase schedule. Total elapsed time is 22 weeks; total effort is roughly 40–46 person-weeks, which fits two engineers at about 60 % allocation.

> **On the overlaps.** Phase 2 starts in week 5 rather than week 7 because the test suite is on the critical path for Phase 3 and has no dependency on Terraform. Phase 4 starts in week 13 because `prom-client` instrumentation is a change to the Express app rather than to the cluster, and shipping it early gives the EKS cut-over a Lambda-era baseline to compare against.

Milestone gates

A gate is passed by demonstration, not by assertion. Each exit criterion below is written so that it can be shown working in a review.

GATE	WEEK	EXIT CRITERIA — ALL MUST BE DEMONSTRABLY TRUE
G0	2	Branch protection active on `main`; docs commit `107129f` rescued and merged; a complete secret and endpoint inventory published; the GitHub OIDC provider and deploy role assumable from a real workflow run.
G1	6	`terraform plan` against production returns **no changes** for all imported resources; remote state in S3 with DynamoDB locking; a staging environment created purely from code.
G2	9	Unit and integration tests run on every pull request with a coverage floor; the API artefact is built in CI and deployed to staging without human hands; `plan` auto-posted to pull requests and `apply` gated on an environment approval.
G3	15	Web and API images in ECR with a passing vulnerability scan; the EKS cluster serving staging end to end; production traffic shifted 10 % through the ALB with a documented rollback.
G4	19	Prometheus scraping both workloads and all cluster components; four golden-signal dashboards live; alert rules paging a real on-call route; two SLOs with error budgets published and burning correctly.
G5	22	Restore from backup rehearsed and timed; WAF in blocking mode; Firebase decommissioned; monthly cost within 10 % of the model in section 12.

Effort distribution

PHASE	PERSON-WEEKS	DOMINANT COST
Phase 0 · Baseline	2	Discovery and documentation, not engineering
Phase 1 · Terraform	8	Import iteration against live resources; the RDS import is the slow part
Phase 2 · Actions	7	Writing the first real test suite from nothing
Phase 3 · EKS	12	Cluster add-ons and the traffic cut-over rehearsals
Phase 4 · Observability	9	Dashboards and alert tuning; the instrumentation itself is a week
Phase 5 · Hardening	8	The auth migration and the disaster-recovery rehearsal
Total	**46**	**Two engineers at ~60 % over 22 weeks**

> **Where the schedule is most likely to slip.** Phase 1's import work is estimated from a resource inventory that does not exist yet, and Phase 3 depends on a decision (§9.1) that has not been made. Treat the 22 weeks as a plan for Phases 0–2, which are well understood, and as a forecast for Phases 3–5, which are not.

PHASE 0

Baseline & guardrails

Weeks 1–2 · 2 person-weeks · Gate **G0** · No new AWS spend

Two weeks of unglamorous work that makes every later phase cheaper. Nothing here changes runtime behaviour; it establishes what exists, who is allowed to change it, and how automation authenticates.

6.1 Rescue and consolidate the repository

- Recover dangling commit `107129f` to a branch and merge it, bringing `docs/security-plan.md` and the peer-coding COP onto `main`.
- Cherry-pick the reverted Docker and `k8s/` work from `6be8279` onto a `feat/containers` branch. It is a head start for Phase 3, not wasted effort.
- Delete or clearly rename the misleading `migratingAuth` branch, and pick one of the three `cloudflare/workers-autoconfig*` branches to keep.
- Correct the stale README: there is no `lambda/` directory, no `npm test` script, and the versions it lists are behind the actual `package.json`.

6.2 Branch protection and repository policy

CONTROL	SETTING
Required pull request reviews	One approval on `main`; stale approvals dismissed on new pushes
Required status checks	`typecheck` and `lint` now; `test` and `terraform-plan` from Phase 2
Linear history	Enforced, squash merges only, so a revert is always exactly one commit
Force push and deletion	Blocked on `main`, `staging` and `production`
Secret scanning	Enabled with push protection; Dependabot for npm, pip, Actions and Docker
Environments	`staging` deploys automatically; `production` requires a reviewer
`CODEOWNERS`	`/terraform/`, `/.github/` and `/k8s/` owned by the platform reviewers

6.3 Configuration and endpoint inventory

Produce the authoritative table of the 13 `VITE_*` variables and 10 server variables, mapping each to its future home: a build-time input, SSM Parameter Store, or Secrets Manager. Appendix A carries the current draft. In the same pass, replace the hard-coded API Gateway hosts in `vite.config.ts`, `src/lib/predict.ts` and `pages.yml` with variables, so that Phase 1 can move endpoints without touching application source.

GitHub OIDC trust and Terraform state bootstrap

The single most valuable artefact of this phase is a keyless path from GitHub Actions into AWS, scoped so that only this repository, and only specific branches and environments, can assume the deploy role.

```
# terraform/bootstrap/main.tf — applied once, by hand, then committed
resource "aws_iam_openid_connect_provider" "github" {
  url             = "https://token.actions.githubusercontent.com"
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

data "aws_iam_policy_document" "assume" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.github.arn]
    }
    condition {
      test     = "StringLike"
      variable = "token.actions.githubusercontent.com:sub"
      # Scope narrowly. Without this condition ANY GitHub repository
      # in the world can assume this role.
      values = [
        "repo:imhero2k/GardenWise:ref:refs/heads/main",
        "repo:imhero2k/GardenWise:environment:staging",
        "repo:imhero2k/GardenWise:environment:production",
      ]
    }
  }
}

# Remote state: a versioned, encrypted bucket plus a lock table.
resource "aws_s3_bucket"      "tfstate" { bucket   = "rootvio-tfstate-apse2" }
resource "aws_dynamodb_table" "tflock"  { hash_key = "LockID" }
```

Deliverables

- Protected `main` with required checks and `CODEOWNERS`
- Documentation and container work rescued onto branches
- Published configuration inventory (Appendix A)
- OIDC provider, deploy role, state bucket, lock table
- `terraform/bootstrap/` committed to the repository

Watch out for

- An over-broad trust policy. Omitting the `sub` condition is the classic OIDC mistake.
- The seven `VITE_FIREBASE_*` values are compiled into a public bundle. They are configuration, not secrets; enforce Firebase security rules instead of hiding them.
- Bootstrap state is chicken-and-egg. Apply locally once, then migrate its own state into the bucket it just created.

Definition of done

CRITERION	VERIFICATION
Documentation is reachable	`git log main -- docs/security-plan.md` returns a commit
CI can reach AWS without keys	A workflow run performs `sts:GetCallerIdentity` using only OIDC
State backend is live	`terraform init` succeeds against the S3 backend with locking

PHASE 1

Terraform foundation

Weeks 3–6 · 8 person-weeks · Gate **G1** · TERRAFORM 1.9 · AWS PROVIDER 5.X

The estate becomes reviewable. The discipline that matters here is **import first, refactor second**: bring every existing resource under management and reach a zero-diff plan before changing a single setting.

7.1 Repository layout

```
terraform/
  bootstrap/                # OIDC provider, state bucket, lock table (Phase 0)
  modules/
    network/                # VPC, 3 AZ public+private subnets, NAT, endpoints, flow logs
    data-postgres/          # RDS PostGIS, subnet group, params, IAM auth, PITR
    data-dynamo/            # planner-layout table, PITR, autoscaling
    api-lambda/             # Phase 1 runtime: Lambda + API Gateway (retired in Phase 3)
    weedscan-lambda/        # container Lambda + ECR repo for the ONNX model
    static-site/            # S3 + CloudFront + ACM + Route 53 (replaces GitHub Pages)
    identity/               # Cognito user pool, clients, hosted UI, IdP federation
    observability/          # CloudWatch groups, alarms, SNS (superseded by Phase 4)
    eks/                    # added in Phase 3
  envs/
    staging/  { main.tf  backend.tf  terraform.tfvars }
    prod/     { main.tf  backend.tf  terraform.tfvars }
```

Environments are deliberately thin: they instantiate modules and set variables, nothing more. Any `if prod` logic inside a module is a smell — lift it to a variable so the two environments stay structurally identical, which is the property that makes staging a meaningful test.

7.2 Import the existing estate

Terraform 1.5 and later support declarative `import` blocks, which are far safer than the old `terraform import` command because the import is reviewed in a pull request and planned before it is executed.

```
# terraform/envs/prod/imports.tf — delete each block once state has settled
import {
  to = module.data_dynamo.aws_dynamodb_table.planner_layout
  id = "rootvio-planner-layout"
}

import {
  to = module.api_lambda.aws_apigatewayv2_api.main
  id = "tcal5txtn8"            # the identifier hard-coded in pages.yml
}

import {
  to = module.weedscan_lambda.aws_apigatewayv2_api.predict
  id = "7muezhf0bl"            # the identifier in src/lib/predict.ts
}

import {
  to = module.data_postgres.aws_db_instance.main
  id = "rootvio-pg"
}

# Then iterate until this is clean:
#   terraform plan -detailed-exitcode        (exit 0 = no changes)
# That zero-diff plan IS gate G1.
```

Week-by-week activity

WK	ACTIVITY	DETAIL AND ACCEPTANCE
3	**Discover and document**	Enumerate the live estate with the AWS CLI: API Gateway stages, Lambda configuration and environment variables, RDS parameters, DynamoDB capacity, IAM roles. Reconcile against the rescued security plan. Output is a resource inventory with ARNs.
3	**Network module**	The VPC is the one thing likely to be genuinely new. Three availability zones, private subnets for RDS and future EKS nodes, a single NAT for staging and one per zone for production, plus gateway endpoints for S3 and DynamoDB to cut NAT egress cost.
4	**Import the data tier**	RDS and DynamoDB under management. Add what is missing: `deletion_protection`, point-in-time recovery, seven-day backup retention, storage encryption, and `prevent_destroy` lifecycle blocks.
4–5	**Import the compute tier**	Both API Gateways and both Lambda functions, including the container-image weed-scan function and its ECR repository. Move all Lambda environment variables to SSM Parameter Store references.
5	**Identity module**	Provision the Cognito user pool, app clients and hosted UI that the `cognito-migration` branch already expects. Provisioned now, cut over in Phase 5 — this decouples infrastructure work from an application migration.
5–6	**Static site module**	S3, CloudFront, ACM and Route 53, giving one origin for the SPA and ending the GitHub Pages / Netlify / Cloudflare fork in the road recorded as gap 10. Keep Pages running in parallel until DNS is cut.
6	**Build staging from code**	The real proof of the phase. `terraform apply` in `envs/staging` must produce a working environment with no console clicks and no manual repair afterwards.

7.3 Conventions worth agreeing before the first module

CONVENTION	RULE
Naming	`rootvio-{env}-{component}`, for example `rootvio-prod-planner-layout`. Set once in a `locals` block and never hand-typed again.
Default tags	Provider-level `default_tags` carrying `Project`, `Environment`, `Owner`, `ManagedBy` and `CostCentre`. This is what makes the cost model in section 12 measurable rather than theoretical.
Version pinning	`required_version = "~> 1.9"`, AWS provider `~> 5.60`, and a committed `.terraform.lock.hcl`. Unpinned providers are how a Tuesday morning becomes an unplanned upgrade.
State isolation	One state file per environment at `s3://rootvio-tfstate-apse2/{env}/terraform.tfstate`. A staging mistake must never be able to reach production state.
Secret handling	No secret values in `.tf` or `.tfvars`. Terraform creates the parameter; a human or a rotation Lambda writes the value. Outputs carrying anything sensitive are marked `sensitive`.
Destroy protection	`lifecycle { prevent_destroy = true }` on RDS, the DynamoDB table, the state bucket and the Route 53 zones.
No inline policies	IAM built from `aws_iam_policy_document` data sources, so intent is reviewable in the plan diff rather than buried in a JSON heredoc.

Keeping drift closed, and knowing when the phase is finished

7.4 Drift control

An infrastructure-as-code adoption fails when somebody fixes production in the console at 11 pm and never tells Terraform. Three mechanisms, in increasing order of teeth:

- **Nightly drift detection.** A scheduled workflow runs `terraform plan -detailed-exitcode` against both environments and opens an issue on exit code 2, so drift becomes visible within a day instead of at the next release.
- **Read-only humans in production.** Once G1 passes, engineers hold a `ReadOnlyAccess` role in the production account. Writes go through the Terraform deploy role that only Actions can assume. A break-glass admin role exists, requires MFA, and raises a CloudTrail alert whenever it is used.
- **Policy as code.** `tflint` and `checkov` in the pull-request pipeline, failing the build on public S3 buckets, unencrypted storage, `0.0.0.0/0` ingress on anything but the load balancer, and IAM wildcards.

> **The database is the risky import.** Some RDS attribute changes force replacement, and a replacement here means data loss. Always read the plan output for `# forces replacement` before applying, take a manual snapshot before the first RDS apply, and rehearse the whole import against a snapshot-restored clone rather than the live instance.

7.5 Definition of done

CRITERION	HOW IT IS VERIFIED
Zero-diff production plan	`terraform plan -detailed-exitcode` in `envs/prod` exits 0, with the output attached to the gate review
Staging built from code alone	A fresh apply into empty state produces a working staging stack, and the smoke test passes against it
Remote state and locking work	Two concurrent applies are attempted; the second blocks on the DynamoDB lock
No hard-coded endpoints remain	`rg 'execute-api'` across `src/`, `vite.config.ts` and `.github/` returns no matches
Backups verified	Point-in-time recovery enabled on RDS and DynamoDB, and a restore to a scratch instance is performed and timed
Cost tags flowing	Cost Explorer can group the estate by `Project` and `Environment`

Effort split

Discovery and inventory	1.5 pw
Network module	1.0 pw
Data tier import	1.5 pw
Compute tier import	1.5 pw
Identity and static site	1.5 pw
Staging build and hardening	1.0 pw

Why Terraform rather than CDK or Pulumi?

Both alternatives are defensible. Terraform is recommended here for three reasons specific to this project: the declarative `import` workflow is the best available tool for adopting a hand-built estate; `terraform-aws-modules/eks` removes weeks of Phase 3 work; and plan output is readable by reviewers who are not fluent in the language it was written in. CDK's main advantage — sharing TypeScript with the application — matters less when the infrastructure surface is this small.

PHASE 2

GitHub Actions delivery

Weeks 5–8 · 7 person-weeks · Gate **G2** · REUSABLE WORKFLOWS · OIDC · ENVIRONMENTS

Today CI proves that the TypeScript compiles. After this phase it proves that the software works, builds every artefact reproducibly, and deploys the whole system without anyone touching the console.

8.1 Target workflow set

WORKFLOW	TRIGGER	RESPONSIBILITY
`ci.yml`	Pull request, push	Install, typecheck, lint, **unit tests**, **API integration tests** against a Postgres service container, coverage floor, bundle-size budget
`terraform.yml`	PR touching `terraform/**`	`fmt`, `validate`, `tflint`, `checkov`, then `plan` per environment posted as a sticky pull-request comment
`terraform-apply.yml`	Merge to `main`	Apply to staging automatically, then to production behind the protected environment's required reviewer
`build-images.yml`	Merge to `main`, tags	Buildx multi-stage builds for web, api and weedscan; push to ECR tagged with the commit SHA; Trivy scan; SBOM attached; image signed with cosign
`deploy-app.yml`	After images	Phase 2 updates the Lambda to the new artefact. Phase 3 bumps the image tag in the Argo CD source and waits for a healthy rollout
`e2e.yml`	Post-deploy to staging	Playwright journeys: sign in, plant search, planner save and reload, weed scan upload
`drift.yml`	Nightly schedule	`plan -detailed-exitcode` against both environments; opens an issue when drift is found
`pages.yml`	—	**Retired** once CloudFront serves the SPA; the multi-branch `iteration1` / `iteration2` preview behaviour is replaced by per-pull-request preview deployments

8.2 Closing the testing gap

Unit — Vitest

Pure logic first, because it is cheap and it is where the real bugs are: `plannerLayoutValidate.mjs` boundary cases, weed-lookup normalisation, recommendation ranking, frost-date arithmetic. Target 70 % statement coverage on `server/`.

Integration — Supertest

All ten Express routes against a real `postgis/postgis` service container seeded from `database/create table.sql`, plus DynamoDB Local. Assert the auth guard: a `PUT /api/planner/layout` without a bearer token must return 401.

End-to-end — Playwright

Four journeys against deployed staging, run post-deploy rather than per-pull-request so they never become the flaky check everyone learns to re-run. Traces and screenshots retained on failure.

> **Sequencing note.** Write the integration tests *before* the Phase 3 container work. They are the only mechanism that will tell you the containerised Express app behaves identically to the Lambda one, and building them afterwards means migrating blind.

Reusable quality workflow

One called workflow holds the quality gate, so `ci.yml`, the nightly job and any future release branch all run byte-identical checks rather than three drifting copies.

```yaml
# .github/workflows/_node-quality.yml — called, not triggered
on:
  workflow_call:
    inputs:
      node-version: { type: string, default: "22" }

jobs:
  quality:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgis/postgis:16-3.4
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: rootvio_test
        options: >-
          --health-cmd pg_isready --health-interval 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "${{ inputs.node-version }}"
          cache: npm
      - run: npm ci
      - run: npm run typecheck
      - run: npm run lint
      - run: npm run test:unit -- --coverage
      - run: npm run test:integration
        env:
          DATABASE_URL: "postgres://postgres:postgres@localhost:5432/rootvio_test"
          DATABASE_SSL: "0"
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: coverage, path: coverage/ }
```

8.3 Supply-chain controls

- **Pin actions by commit SHA**, not by tag. A moved tag on a third-party action is a credible supply-chain vector, and this pipeline holds a role that can write to production.
- **Least-privilege** `permissions:` declared per job. The default token is read-only, and `id-token: write` appears only on jobs that actually assume a role.
- **Concurrency groups** per environment with `cancel-in-progress: false`, so two applies can never race for the state lock.
- **Trivy and cosign** on every image; the deploy job refuses an unsigned image or one carrying an unwaived critical CVE.
- **Reproducible builds** from `npm ci` against a committed lockfile, with the commit SHA as the only image tag. No `latest`, ever.

> **Watch the runner minutes.** The current `pages.yml` builds three branch variants on every push to any of them, so a single change can trigger nine builds. Splitting previews into per-pull-request deployments and caching `node_modules` and the Playwright browsers typically cuts total pipeline time by well over half.

Keyless deploys and gated applies

```yaml
# .github/workflows/terraform.yml — plan stage
permissions:
  id-token: write        # required for OIDC
  contents: read
  pull-requests: write

jobs:
  plan:
    runs-on: ubuntu-latest
    strategy:
      matrix: { env: [staging, prod] }
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4    # no static keys anywhere
        with:
          role-to-assume: arn:aws:iam::${{ secrets.AWS_ACCOUNT_ID }}:role/rootvio-gha-plan
          aws-region: ap-southeast-2
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.9.8" }
      - run: terraform init -input=false
        working-directory: terraform/envs/${{ matrix.env }}
      - run: terraform plan -no-color -input=false -out=tfplan | tee plan.txt
        working-directory: terraform/envs/${{ matrix.env }}
      - uses: marocchino/sticky-pull-request-comment@v2   # reviewers see the diff
        with:
          header: "plan-${{ matrix.env }}"
          path: terraform/envs/${{ matrix.env }}/plan.txt
```

ROLE	ASSUMABLE FROM	PERMISSIONS
`rootvio-gha-plan`	Any pull-request ref	`ReadOnlyAccess` plus read and write on the state bucket and lock table. It cannot mutate infrastructure, so a fork pull request cannot do damage.
`rootvio-gha-apply-staging`	`environment:staging`	Write access scoped to staging resources by tag and name-prefix condition
`rootvio-gha-apply-prod`	`environment:production`	Write access to production, reachable only after the environment's required reviewer approves
`rootvio-gha-ecr-push`	`ref:refs/heads/main`	`ecr:PutImage` and layer upload on the three repositories, and nothing else

Why three roles rather than one. A single powerful role assumable from any workflow means a pull request from a fork, or a compromised third-party action, inherits the ability to write to production. Splitting plan from apply, and staging from production, means the blast radius of each credential matches the job that holds it.

Definition of done

CRITERION	VERIFICATION
The test suite gates merges	Unit and integration checks required on `main`; a deliberately broken SQL query fails the pull request
Coverage floor enforced	The build fails below 70 % on `server/`, and the trend is published per pull request
The API deploys without hands	A merge to `main` reaches staging automatically; production waits for the environment approval
No static AWS credentials	Zero `AWS_SECRET_ACCESS_KEY` secrets in repository settings; CloudTrail shows only `AssumeRoleWithWebIdentity`
Plans visible to reviewers	Every `terraform/**` pull request carries a sticky plan comment for both environments
Rollback rehearsed	A previous artefact is redeployed on purpose and timed; the target is under ten minutes

Keep Pages until CloudFront is proven. Retire `pages.yml` only after the CloudFront distribution has served production traffic for a week. The custom domain `rootivio.app` and the `CNAME` file are the cut-over point, so plan for DNS TTL and keep the Pages rollback ready.

PHASE 3

Containers & Amazon EKS

Weeks 9–14 · 12 person-weeks · Gate **G3** · **EKS 1.31 · KARPENTER · ARGO CD**

9.1 Decide this before week 9

> **EKS is the largest cost and complexity increase in this roadmap** — roughly US$ 150–220 per month per environment, plus a permanent operational surface. If the driver is a learning objective or a coursework requirement, proceed as written. If the driver is purely production need at current traffic, Lambda with API Gateway is honestly the better fit, and the Terraform and Actions work in Phases 1–2 delivers most of the benefit without it. Phase 4's Prometheus stack does assume a cluster; the ECS Fargate alternative in §9.6 is the middle path.

9.2 Recover and finish the container work

Commit `6be8279` already contains `Dockerfile`, `Dockerfile.api`, `docker-compose.yml`, `nginx.conf` and four `k8s/` manifests. Start from the cherry-pick made in Phase 0 rather than a blank page, then harden it.

```
# Dockerfile — web: build the SPA, serve it from a minimal nginx
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_BASE_URL VITE_COGNITO_USER_POOL_ID VITE_BASE_PATH=/
RUN npm run build

FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
USER nginx                                  # never run as root
HEALTHCHECK CMD wget -qO- http://127.0.0.1:8080/healthz || exit 1

# Dockerfile.api — production dependencies only, non-root, no shell tooling
FROM node:22-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:22-alpine
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY server ./server
ENV NODE_ENV=production PORT=3001
USER node
EXPOSE 3001
CMD ["node", "server/index.mjs"]
```

One required code change, and the cluster shape

9.3 Split the probe surface

`server/app.mjs` currently mounts everything on a single Express app that Lambda wraps. For Kubernetes the probe surface has to be split, so that a saturated database does not cause the kubelet to restart otherwise-healthy pods:

- `GET /healthz` — liveness. Returns 200 if the process is responsive. **No database check.**
- `GET /readyz` — readiness. Checks the `pg` pool and DynamoDB reachability; failing removes the pod from the Service but does not restart it.
- `GET /api/health` — kept as the existing rich, human-facing status payload.
- `GET /metrics` — added in Phase 4, bound to the pod network only.

The API also needs graceful `SIGTERM` handling in `server/index.mjs`, so that a rolling update drains in-flight requests instead of dropping them, and JSON logging to stdout so that Loki can parse it in Phase 4.

9.4 Cluster shape

DECISION	CHOICE	REASONING
Provisioning	`terraform-aws-modules/eks` v20	Handles IRSA, add-ons and access entries, saving weeks versus hand-rolled resources
Version	EKS 1.31	One minor behind latest, with an upgrade cadence set at two per year
Compute	Karpenter on `t4g` / `m7g` Graviton spot, on-demand fallback	Roughly 20 % cheaper per vCPU than x86, and it bin-packs far better than fixed node groups
Networking	VPC CNI, private nodes, public ALB only	Nodes are never reachable from the internet
Ingress	AWS Load Balancer Controller with external-dns	One ALB for both workloads; DNS records follow the Ingress objects
Secrets	External Secrets Operator reading SSM and Secrets Manager	No secret material in Git; Terraform owns the parameter, the operator syncs the value
Delivery	Argo CD, app-of-apps pattern	Cluster state is reconciled from the repository, and drift self-heals
Access	EKS access entries, IAM-mapped, no `aws-auth` edits	The modern mechanism; avoids the classic ConfigMap lockout

Week-by-week activity

WK	ACTIVITY	DETAIL
9	**Images and ECR**	Finish the three Dockerfiles, create ECR repositories in Terraform with lifecycle policies and scan-on-push, and wire `build-images.yml`. Verify with `docker compose up` that the local stack still works.
10	**Probes and configuration**	Split `/healthz` and `/readyz`, move all configuration to environment variables with no filesystem assumptions, and switch to JSON logging.
10–11	**EKS module, staging**	Cluster, Karpenter node pools, IRSA roles for DynamoDB and RDS IAM auth, and the core add-ons: CoreDNS, kube-proxy, VPC CNI, EBS CSI.
11–12	**Platform add-ons**	AWS Load Balancer Controller, external-dns, External Secrets Operator, metrics-server, and Argo CD with the app-of-apps root.
12	**Helm chart**	One chart, two value files: Deployments for web and api, Services, a shared Ingress, HPAs, PodDisruptionBudgets, resource requests and limits, topology spread across three zones, and a `NetworkPolicy` restricting api ingress to the web pods and the ALB.
13	**Staging end to end**	The Playwright suite green against the EKS-hosted staging URL, plus a load test to size requests and limits from evidence rather than guesswork.
13–14	**Production and canary**	Apply the same module to production, then weight the ALB target groups 90/10 and watch for 48 hours. **This is gate G3.**

Traffic cut-over

STEP	SPLIT	HOLD	ABORT CONDITION
Canary	90 Lambda / 10 EKS	48 hours	5xx rate above 0.5 %, or p95 latency more than 20 % worse than the Lambda baseline
Half	50 / 50	72 hours	Any Sev-1 incident, or database connections above 80 % of the pool ceiling
Majority	10 / 90	1 week	Error-budget burn faster than twice the target rate
Complete	0 / 100	—	Lambda and API Gateway kept warm and deployable for 30 more days before removal

The connection-pool trap. One Lambda concurrency unit holds one `pg` connection, but twelve api pods each holding a pool of ten is 120 connections, and a `db.t4g.micro` caps out well below that. Before the canary, either put RDS Proxy in front of Postgres or set the pool `max` to `ceil(db_max_connections × 0.8 / max_replicas)`. This is the most common way a Lambda-to-Kubernetes migration falls over on day one, and it fails under load rather than in staging.

Definition of done

- Three images in ECR, signed, scanned and tagged by commit SHA, with no unwaived critical CVEs.
- Staging and production clusters created entirely by Terraform; a cluster can be destroyed and rebuilt in under an hour.
- Argo CD reconciling, demonstrated by a manual `kubectl edit` against a managed object being reverted automatically.
- HPAs demonstrably scaling under load test, and PodDisruptionBudgets holding availability through a node drain.
- Production serving at least 10 % of live traffic from EKS with a written, rehearsed rollback.

9.6 If EKS is not justified — the ECS Fargate variant

Should the week-9 decision go the other way, most of this phase still applies: the Dockerfiles, the ECR repositories, the probe split, JSON logging and GitOps-style deploys are all unchanged. Swap the `eks/` module for an `ecs-fargate/` module, and in Phase 4 replace kube-prometheus-stack with Amazon Managed Service for Prometheus scraping the same `/metrics` endpoint through an ADOT sidecar. That path is roughly 60 % cheaper and materially less to operate; the trade-off is losing the Kubernetes ecosystem and the learning outcome.

9.7 The weedscan decision

The ONNX inference Lambda is bursty and idles at zero, which is precisely the workload shape AWS Lambda is best at. Keeping it there is recommended. Moving it to a KEDA-scaled Kubernetes job is possible and is drawn that way in Figure 1, but it should be done because queue-depth scaling or GPU scheduling is genuinely needed, not for the sake of uniformity.

PHASE 4

Observability with Prometheus

Weeks 13–18 · 9 person-weeks · Gate **G4** · KUBE-PROMETHEUS-STACK · GRAFANA · LOKI · TEMPO

The system currently emits one boolean health endpoint and unstructured console output. This phase makes it answer three questions it cannot answer today: *is it broken*, *for whom*, and *why*.

10.1 Stack composition

COMPONENT	DEPLOYMENT	PURPOSE
Prometheus Operator	`kube-prometheus-stack` chart, via Argo CD	CRD-driven scrape configuration, so `ServiceMonitor` objects live beside the workloads they describe
Prometheus	2 replicas, 50 GiB gp3 each	15-day local retention at a 30-second scrape interval
Thanos or AMP	Optional, week 18	Long-term retention, only if 15 days proves insufficient for capacity planning
Grafana	Dashboards as ConfigMaps, sidecar-loaded	Dashboards are code and are reviewed like code; SSO for access
Alertmanager	3 replicas with HA gossip	Routes by severity: `critical` pages, `warning` goes to Slack
Exporters	node-exporter, kube-state-metrics, `postgres_exporter`, CloudWatch exporter	Node, cluster, database and AWS-managed-service signals all in one query language
Loki + Promtail	Loki single-binary, S3 backend	Structured logs correlated to metrics by `trace_id`
OTel Collector + Tempo	DaemonSet collector, Tempo on S3	Traces across HTTP, SQL and DynamoDB, linked from Prometheus exemplars

10.2 Rollout order

WEEK	ACTIVITY
13	`prom-client` shipped while the API is still on Lambda, capturing the pre-migration baseline
14	kube-prometheus-stack deployed to staging through Argo CD
15	ServiceMonitors, exporters, and the first two dashboards
16	Alert rules, Alertmanager routes, and a runbook per critical alert
17	Loki and Promtail; JSON logs correlated by trace identifier
18	OpenTelemetry and Tempo, exemplars, SLO dashboards. **Gate G4.**

Instrumenting the Express API

This change is small and is the highest-leverage work in the phase. Ship it in week 13, while the API is still on Lambda, so the EKS cut-over has a genuine before-and-after baseline.

```
// server/metrics.mjs — new file
import client from "prom-client";

export const registry = new client.Registry();
registry.setDefaultLabels({ service: "rootvio-api", env: process.env.APP_ENV });
client.collectDefaultMetrics({ register: registry });   // event loop, GC, heap, RSS
export const httpDuration = new client.Histogram({
  name: "http_request_duration_seconds",
  help: "HTTP request latency",
  labelNames: ["method", "route", "status"],
  // buckets chosen around the current p95, not the library defaults
  buckets: [0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5],
});

export const dbQueryDuration = new client.Histogram({
  name: "db_query_duration_seconds", help: "Postgres query latency",
  labelNames: ["query", "outcome"], buckets: [0.005, 0.02, 0.1, 0.5, 2, 10],
});

// the metric that predicts the §9 pool exhaustion before users feel it
export const dbPoolInUse = new client.Gauge({
  name: "db_pool_connections_in_use", help: "Busy pg pool connections",
});

// the provider label makes the Firebase -> Cognito cut-over observable
export const authFailures = new client.Counter({
  name: "auth_verification_failures_total", help: "Rejected tokens",
  labelNames: ["provider", "reason"],
});

[httpDuration, dbQueryDuration, dbPoolInUse, authFailures]
  .forEach((m) => registry.registerMetric(m));
```

```
// server/app.mjs — middleware, mounted before the routes
app.use((req, res, next) => {
  const end = httpDuration.startTimer();
  res.on("finish", () => end({
    method: req.method,
    // the route PATTERN, never req.path: /api/plants/:id must not
    // become thousands of distinct label values
    route: req.route?.path ?? req.baseUrl ?? "unmatched",
    status: res.statusCode,
  }));
  next();
});

app.get("/metrics", async (_req, res) => {
  res.set("Content-Type", registry.contentType);
  res.end(await registry.metrics());
});
```

> **Cardinality is the one way to break Prometheus.** Every distinct label combination is a separate time series. Never label with a user identifier, a raw path or a full error message. Ten routes by five methods by six status codes is a few thousand series and entirely fine; adding `userId` turns it into millions and takes the server down.

Scrape targets and alert rules as Kubernetes objects

```yaml
# k8s/observability/servicemonitor-api.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: rootvio-api
  namespace: rootvio
  labels: { release: kube-prom-stack }
spec:
  selector:
    matchLabels: { app.kubernetes.io/name: rootvio-api }
  endpoints:
    - port: http
      path: /metrics
      interval: 30s
      relabelings:                    # keep pod identity for drill-down
        - sourceLabels: [__meta_kubernetes_pod_name]
          targetLabel: pod
```

```yaml
# k8s/observability/prometheusrule-slo.yaml — symptom-based, not cause-based
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
spec:
  groups:
    - name: rootvio-api.rules
      rules:
        - alert: ApiHighErrorRate
          expr: |
            sum(rate(http_request_duration_seconds_count{status=~"5.."}[5m]))
              / sum(rate(http_request_duration_seconds_count[5m])) > 0.02
          for: 10m
          labels: { severity: critical }
          annotations:
            summary: "API 5xx above 2% for 10 minutes"
            runbook: "https://github.com/imhero2k/GardenWise/blob/main/docs/runbooks/api-5xx.md"

        - alert: ApiLatencyBudgetBurn
          expr: |
            histogram_quantile(0.95,
              sum by (le) (rate(http_request_duration_seconds_bucket[5m]))) > 1.0
          for: 15m
          labels: { severity: warning }

        # the §9 failure mode, caught before users feel it
        - alert: DbPoolNearExhaustion
          expr: db_pool_connections_in_use / db_pool_connections_max > 0.85
          for: 5m
          labels: { severity: critical }

        # the only write path in the product
        - alert: PlannerLayoutSaveFailing
          expr: |
            sum(rate(http_request_duration_seconds_count{
              route="/api/planner/layout", method="PUT", status=~"5.."}[10m])) > 0
          for: 10m
          labels: { severity: critical }
```

Dashboards and service level objectives

10.3 Dashboards to build, in this order

DASHBOARD	PANELS
API golden signals	Request rate by route, error rate by status class, p50/p95/p99 latency, in-flight requests, saturation against the HPA target
Data tier	Pool in-use versus max, query latency by statement, slow-query count, DynamoDB consumed capacity, throttles and latency for the layout table
Product journeys	Sign-ins by provider, planner saves and load failures, weed-scan invocations and inference latency, PlantNet upstream error rate
Cluster health	Node CPU, memory and disk pressure, pod restarts and OOMKills, pending pods, Karpenter provisioning latency, spot interruption events
SLO & error budget	Multi-window burn rate, budget remaining this period, and a deploy-annotation overlay so a regression is visibly tied to a release

10.4 Service level objectives

SERVICE LEVEL INDICATOR	OBJECTIVE	WINDOW	ERROR BUDGET
API availability — non-5xx over total requests	99.5 %	30 days rolling	3 h 39 m
API latency — requests served under 500 ms	95 %	30 days rolling	5 % of requests
Planner-layout save success	99.9 %	30 days rolling	44 minutes
SPA availability at the edge	99.9 %	30 days rolling	44 minutes

Alert on multi-window burn rate rather than on raw thresholds: a 14.4× burn over one hour pages, and a 6× burn over six hours raises a ticket. Threshold alerts on a low-traffic service page constantly at 3 am for nothing, and a pager that people learn to ignore is worse than no pager at all.

Definition of done

Observability is the phase where "we set it up" and "it works" diverge most often, so every criterion below is written as something that has to be demonstrated against a real failure.

10.5 Exit criteria

CRITERION	VERIFICATION
All scrape targets healthy	The Prometheus targets page shows `up` for api, web, node-exporter, kube-state-metrics, postgres-exporter and the CloudWatch exporter
Dashboards provisioned from Git	Five dashboards load from ConfigMaps; none has been hand-edited in the Grafana UI
Alerts reach a human	An induced failure in staging delivers a page through the real on-call route, not a test webhook
SLOs published and burning	Four SLOs with visible burn-rate panels, and the arithmetic checked against a known incident
Runbooks exist	Every `severity: critical` rule carries a `runbook` annotation that resolves to a real document
An incident can be traced	One incident replayed end to end: alert, dashboard, log line, trace, root cause

> **The order matters more than the tooling.** Instrumentation before dashboards, dashboards before alerts, alerts before SLOs. Teams that start with SLOs end up with objectives that nothing measures, and teams that start with alerts end up paging on causes rather than on symptoms users can actually feel.

PHASE 5

Hardening, DR & FinOps

Weeks 17–22 · 8 person-weeks · Gate **G5**

With the platform codified, delivered and observed, the last phase turns it into something that can be operated by someone who did not build it.

11.1 Security posture

AREA	WORK
Finish the auth migration	Merge `cognito-migration`, run the dual-verify window with the `authFailures{provider}` metric proving the traffic shift, then remove Firebase Admin, the seven `VITE_FIREBASE_*` secrets and the `firebase` dependency. This closes gap 7.
API authorisation	Eight of the ten routes are unauthenticated today. Decide deliberately which stay public, and put API Gateway or WAF rate limiting in front of the expensive ones — particularly `POST /api/plantnet/identify`, which spends money with a third-party API on every call.
WAF	Managed rule groups plus a rate-based rule. Run in count mode for two weeks, review the false positives, then switch to blocking.
Network policy	Default-deny in the `rootvio` namespace, explicitly allowing web to api, api to RDS, and api to the DynamoDB endpoint only.
Database credentials	Switch from a password embedded in `DATABASE_URL` to RDS IAM authentication through IRSA, removing the last long-lived credential from the runtime.
Pod security	The `restricted` Pod Security Standard, a read-only root filesystem, dropped capabilities and seccomp profiles.
Audit	A CloudTrail organisation trail, GuardDuty, Security Hub, and EKS control-plane audit logs shipped to Loki.

11.2 Resilience and disaster recovery

SCENARIO	RTO	RPO	MECHANISM, AND HOW IT IS PROVEN
Single pod failure	seconds	0	Replicas plus readiness probes; proven by deleting a pod under load
Node or zone loss	< 5 min	0	Topology spread across three zones, PDBs, Karpenter replacement; proven by draining a node
Bad release	< 10 min	0	`kubectl rollout undo` or an Argo CD revert; rehearsed at gate G2
Database corruption	< 2 h	< 5 min	RDS point-in-time restore to a new instance, then repoint; **rehearsed and timed at G5**
Accidental table deletion	< 1 h	< 5 min	DynamoDB PITR plus `prevent_destroy`; restore rehearsed
Cluster loss	< 4 h	0	Terraform rebuild plus Argo CD reconciliation; rehearsed on staging
Region loss	Accepted	< 24 h	Out of scope. Cross-region snapshot copies only, with the gap documented and formally accepted.

> **A backup you have not restored is a hypothesis.** The single most valuable hour in this phase is performing an actual point-in-time restore, timing it, and writing down every step that was not obvious. Do it on staging first, then on production off-peak.

Cost governance and operational readiness

11.3 FinOps

- **Right-size from evidence.** Phase 4 provides four weeks of real CPU and memory data; set requests at observed p95 usage and limits at roughly twice requests, instead of guessing at both.
- **Graviton and spot.** Karpenter on `t4g` and `m7g` spot capacity for the stateless web and api pods, with an on-demand fallback pool for observability workloads that should not be interrupted mid-scrape.
- **Scale staging to zero overnight.** A scheduled workflow drops staging replicas outside working hours — typically 60 % off the staging bill for one afternoon of work.
- **Kill NAT egress.** Gateway endpoints for S3 and DynamoDB, and interface endpoints for ECR, SSM and Secrets Manager. NAT data processing is frequently the largest surprise on a first EKS invoice.
- **Budgets and anomaly detection.** AWS Budgets at 80 % and 100 % of the section 12 model, with Cost Anomaly Detection alerting into the same Slack channel as Alertmanager.
- **Compute Savings Plan** once three months of steady-state usage exists — not before, because committing early to the wrong baseline is worse than paying on demand.

11.4 Operational readiness

The deliverable is a handover pack, on the assumption that the people operating this platform in a year are not the people building it now:

ARTEFACT	CONTENT
Architecture decision records	One per significant choice in this document, including the ones that were rejected and why — EKS versus ECS, Terraform versus CDK, Cognito versus Firebase
Runbooks	One per critical alert, linked from the alert's `runbook` annotation, each with a diagnosis path and a rollback step
On-call rotation	A named rotation with escalation, or an explicit written decision that alerts are business-hours best-effort
Incident process	Severity definitions, a communication channel, and a blameless review template
Upgrade calendar	Quarterly EKS and add-on upgrades, plus provider version bumps, scheduled rather than deferred
Corrected README	The current one references a `lambda/` directory and an `npm test` script that do not exist; the architecture diagram needs to match Figure 1

11.5 Definition of done

CRITERION	VERIFICATION
Restore proven, not assumed	A point-in-time restore is performed and the elapsed time recorded against the stated RTO
WAF blocking	Rules moved out of count mode, with the false-positive review documented
Firebase gone	No `firebase` dependency in `package.json` and no Firebase secrets in repository settings
No long-lived database password	The API authenticates to RDS with an IAM token issued through IRSA
Cost within model	The monthly bill lands within 10 % of section 12, with variances explained
Someone else can operate it	An engineer who did not build the platform resolves a staged incident using only the runbooks

Monthly AWS run-rate, ap-southeast-2, USD

List prices at the time of writing, covering both environments where relevant. Excludes GitHub Actions minutes, which the free tier likely covers at this scale, and excludes engineering time.

COMPONENT	TODAY	AFTER P1	AFTER P3–P4	NOTE
EKS control plane × 2 environments	—	—	146	$0.10 per hour per cluster, fixed regardless of load
Worker nodes	—	—	95	Graviton spot, 2–4 nodes; roughly $210 on demand
Observability nodes and EBS	—	—	62	Prometheus, Grafana, Loki and Tempo
Application Load Balancer	—	—	26	One shared ALB plus capacity units
NAT gateway and data	—	36	48	Single NAT in staging; VPC endpoints cut this materially
RDS PostgreSQL	18	46	46	Multi-AZ from Phase 1; this is the Phase 1 delta
DynamoDB	2	3	3	On-demand capacity plus point-in-time recovery
Lambda and API Gateway	6	6	2	Weedscan stays on Lambda; the API tier retires
CloudFront, S3 and WAF	0	14	16	Replaces GitHub Pages hosting
ECR, Secrets Manager, CloudTrail	0	5	11	Storage and API call volume
Cognito	0	0	0	Free below 50,000 monthly active users
Total per month	**~26**	**~110**	**~455**	**~185 on the ECS Fargate variant**

The EKS jump is not incidental. The control plane and the always-on observability stack account for well over half the post-Phase-4 figure, and both are fixed costs that do not fall when traffic does. This is precisely why the week-9 decision in §9.1 matters more than any other single choice in this document.

Levers, in order of return

Choose ECS Fargate over EKS	−$270
Single cluster, namespaced environments	−$73
Graviton spot rather than on-demand	−$115
Scale staging to zero overnight	−$40
VPC endpoints instead of NAT egress	−$25
Single-AZ RDS in staging	−$14

Applying the first lever alone brings the target-state run-rate to roughly $185 per month while preserving every other outcome in this roadmap: Terraform-managed infrastructure, automated delivery, containerised workloads and Prometheus observability. The only thing lost is Kubernetes itself.

Ten risks worth tracking

Ordered by expected loss rather than by phase. Each mitigation is an action already scheduled somewhere in this roadmap, not a general intention.

#	RISK	LIKELY	IMPACT	MITIGATION
R1	**Terraform import damages production data**	Medium	Critical	Rehearse against a snapshot-restored clone; take a manual snapshot before the first RDS apply; scan every plan for `forces replacement`; `prevent_destroy` on all stateful resources
R2	**Connection-pool exhaustion at cut-over**	High	High	RDS Proxy or a computed per-pod pool ceiling before the canary; ship the `DbPoolNearExhaustion` alert first; hold the canary at 10 % for 48 hours
R3	**EKS cost and complexity outrun the team**	High	High	Make the week-9 go/no-go explicit; keep the ECS Fargate variant in §9.6 live as a fallback; budget alerts from day one
R4	**Console drift reopens after gate G1**	High	Medium	Nightly drift workflow; read-only human access in production; an audited break-glass role
R5	**Auth migration strands users**	Medium	High	Dual-verify window; per-provider failure metric as the cut-over gate; a documented rollback to Firebase-only
R6	**Metric cardinality overwhelms Prometheus**	Medium	Medium	Route patterns only, never raw paths or identifiers; `sample_limit` on ServiceMonitors; alert on time-series growth
R7	**Two engineers at 60 % cannot hold 22 weeks**	Medium	Medium	Phases 0–2 deliver standalone value and are the true priority; Phases 3–5 can slip without wasting earlier work
R8	**Dangling docs commit garbage-collected**	Medium	Low	Rescue `107129f` to a branch in week 1 — the first task in Phase 0
R9	**Domain cut-over breaks the live site**	Low	High	Lower the DNS TTL a week ahead; run CloudFront and Pages in parallel; hold the `CNAME` rollback ready
R10	**Third-party quota exhaustion**	Medium	Low	The PlantNet key is public in the bundle today; proxy it server-side, cache responses and rate-limit per user

Review cadence

Re-score this register at each milestone gate. R2 and R3 should be re-assessed specifically at the week-9 decision point, since both are contingent on it, and R7 should be re-assessed at gate G2, which is the earliest point where actual velocity can be compared against the estimate in section 05.

Configuration migration — where each variable ends up

Produced in Phase 0 and maintained thereafter. Anything marked as a build argument is compiled into the public bundle and must never hold a secret.

VARIABLE	TODAY	TARGET HOME	NOTE
`VITE_API_BASE_URL`	CI literal	Terraform output → build arg	Removes the `tcal5txtn8` hard-code
`VITE_PREDICT_API_URL`	Source literal	Terraform output → build arg	Removes the `7muezhf0bl` hard-code
`VITE_FIREBASE_*` (×7)	GitHub secrets	Deleted in Phase 5	Public in the bundle; not secrets
`VITE_COGNITO_*` (×3)	Branch only	Terraform output → build arg	From the `identity` module
`VITE_PLANTNET_API_KEY`	GitHub secret	Server-side only	Proxy through the API; stop shipping it to browsers
`VITE_GOOGLE_MAPS_API_KEY`	Local	Build arg + referrer restriction	Restrict the key at the provider
`VITE_BASE_PATH`	CI literal	Build arg	Becomes `/` once CloudFront replaces Pages
`VITE_BYPASS_AUTH`	Local	Never set outside dev	Add a CI guard that fails the build if truthy
`DATABASE_URL`	Lambda env var	Secrets Manager, then RDS IAM auth	Rotated; the password is removed entirely in Phase 5
`DATABASE_SSL`, `DATABASE_SSL_REJECT_UNAUTH`	Lambda env vars	Helm values	TLS always on outside local development
`DYNAMODB_PLANNER_LAYOUT_TABLE`	Lambda env var	Terraform output → ConfigMap	The table name stops being a magic string
`FIREBASE_SERVICE_ACCOUNT_JSON`	Lambda env var	Secrets Manager, then deleted	Gone once Cognito is the only provider
`PLANTNET_API_KEY`	Lambda env var	Secrets Manager	The server-side counterpart of the public key above
`CORS_ORIGIN`	Lambda env var	Helm values per environment	An explicit allow-list, never a wildcard
`AWS_REGION`, `PORT`	Defaults	ConfigMap	No behavioural change

Repository files this roadmap touches

A change map for reviewers: which parts of the codebase each phase reaches into, and what kind of change it makes.

PATH	PHASE	CHANGE
`terraform/**`	0, 1, 3	New. Bootstrap first, then modules and environments
`.github/workflows/**`	2	Eight workflows replace the current two; `pages.yml` retired
`server/app.mjs`	3, 4	Probe split, metrics middleware, JSON logging
`server/metrics.mjs`	4	New. The `prom-client` registry and instruments
`server/index.mjs`	3	Graceful `SIGTERM` shutdown so rollouts do not drop requests
`server/*.mjs` query modules	2, 4	Unit tests, plus `dbQueryDuration` timers
`server/cognitoAuth.mjs`	5	Merged from `cognito-migration`; the Firebase path is removed after cut-over
`Dockerfile`, `Dockerfile.api`, `nginx.conf`	3	Restored from `6be8279` and hardened
`k8s/**`, `charts/rootvio/**`	3, 4	Helm chart, ServiceMonitors, PrometheusRules, dashboards
`vite.config.ts`, `src/lib/predict.ts`	0, 1	Hard-coded API Gateway hosts replaced with variables
`tests/**`	2	New. Vitest unit, Supertest integration, Playwright end-to-end
`docs/adr/**`, `docs/runbooks/**`	all	New. Decision records and one runbook per critical alert
`README.md`	0, 5	Corrected; the stale `lambda/` and `npm test` claims removed

Where to start on Monday

Hour one

`git branch rescue/docs 107129f`. The security plan is the only written record of the intended AWS topology, and it is currently unreachable from any branch.

Day one

Turn on branch protection and secret scanning, and cherry-pick the reverted container work onto `feat/containers` so Phase 3 does not start from a blank page.

Week one

Enumerate the live AWS estate with the CLI and write down every ARN. Phase 1 cannot be estimated, let alone executed, without that inventory.

A closing note on sequencing. The temptation with a roadmap like this is to start with the most interesting phase. Resist it. Phases 0–2 remove all four critical gaps, cost seventeen of the forty-six person-weeks, and add roughly $84 per month. Phase 3 adds $345 per month and a permanent operational surface. Do the cheap, high-value work first, and let the evidence from Phase 4's metrics inform whether Phase 3 was the right call after all.

Open decisions

Each of these blocks a phase. The default column is what will happen if no decision is taken, so that the absence of a decision does not become an absence of progress.

#	QUESTION	NEEDED BY	DEFAULT IF UNANSWERED
D1	Is EKS a requirement in itself, or is production fit the goal?	Week 9	Proceed with EKS as written, holding §9.6 as the fallback
D2	One AWS account, or separate staging and production accounts?	Week 3	One account with two VPCs and strict tag-scoped IAM. Separate accounts are better practice but add Organizations overhead disproportionate to the team size
D3	Cognito or Firebase as the final identity provider?	Week 5	Cognito — the branch work already exists and it keeps identity inside the Terraform-managed estate
D4	Retire GitHub Pages in favour of CloudFront?	Week 6	Yes. One hosting story, and it ends the Pages / Netlify / Cloudflare fork recorded as gap 10
D5	Self-hosted Prometheus or Amazon Managed Prometheus?	Week 13	Self-hosted `kube-prometheus-stack` — cheaper at this scale and a better learning outcome
D6	Move the weedscan ONNX Lambda into the cluster?	Week 14	No. Bursty scale-to-zero inference is the workload Lambda is genuinely best at
D7	Who carries the pager, and during what hours?	Week 16	Business-hours best-effort, with Alertmanager routing to Slack only until a real rotation exists
D8	Which of the three `cloudflare/*` branches survives?	Week 2	Delete all three once D4 is settled in favour of CloudFront

How a decision gets recorded

Each decision above becomes an architecture decision record in `docs/adr/` once taken, using the standard four-part form: context, the options considered, the decision, and the consequences accepted. The value is almost entirely in the last two sections — in a year, nobody will remember why EKS was chosen over ECS Fargate, and the cost difference in section 12 makes that a question somebody will eventually ask.

Decisions D1 and D3 are the two worth writing up in detail. D1 sets the cost and operational profile of the entire platform, and D3 determines whether two identity providers have to be secured indefinitely or only during a bounded migration window.